



Solutions-architect-pro

AWS SA Professional (SAP-C02)

Curso completo com 7 módulos

Complexidade • Novas Soluções • Melhoria • Migração • Cenários

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim

Visão Geral do Exame

Detalhes do exame SAP-C02, distribuição de domínios, Well-Architected Framework, perfil do candidato e dicas estratégicas para aprovação.

Detalhes do Exame SAP-C02

Item	Detalhe
Código	SAP-C02
Questões	75 (65 pontuadas + 10 não pontuadas)
Duração	180 minutos
Nota de Aprovação	750 / 1000
Custo	USD \$300
Formato	Múltipla escolha e múltipla resposta
Idiomas	Inglês, Japonês, Coreano, Chinês Simplificado
Validade	3 anos
Pré-requisito recomendado	2+ anos de experiência prática com AWS

Distribuição de Domínios

O exame é dividido em 4 domínios com pesos diferentes:

Domínio	Descrição	Peso
Domínio 1	Design de Soluções para Complexidade Organizacional	26%
Domínio 2	Design para Novas Soluções	29%
Domínio 3	Melhoria Contínua de Soluções Existentes	25%
Domínio 4	Aceleração da Migração e Modernização de Workloads	20%

 **Estratégia:** Domínio 2 (Novas Soluções) é o mais pesado com 29%. Foque em arquiteturas serverless, event-driven e DR strategies. Domínios 1 e 3 juntos somam 51% — networking e

otimização são fundamentais.

Well-Architected Framework — 6 Pilares

O framework é a base conceitual do exame. Todas as questões avaliam decisões arquiteturais sob a ótica destes pilares:

Excelência Operacional **PILAR 1**

Executar e monitorar sistemas para entregar valor de negócio e melhorar continuamente processos e procedimentos.

Práticas-chave:

- IaC (CloudFormation, Terraform)
- Observabilidade (CloudWatch, X-Ray)
- Automação de operações
- Runbooks e playbooks
- Deploys frequentes e pequenos

Segurança **PILAR 2**

Proteger informações, sistemas e ativos enquanto entrega valor de negócio através de avaliação de risco e estratégias de mitigação.

Práticas-chave:

- Princípio do menor privilégio
- Rastreabilidade (CloudTrail)
- Segurança em todas as camadas
- Automação de segurança
- Proteção de dados em trânsito e repouso
- Preparação para incidentes

Confiabilidade **PILAR 3**

Garantir que um workload execute sua função pretendida corretamente e consistentemente quando esperado.

Práticas-chave:

- Recuperação automática de falhas
- Teste de procedimentos de recuperação
- Escala horizontal
- Eliminar pontos únicos de falha
- Gerenciamento de mudanças automatizado

Eficiência de Performance **PILAR 4**

Usar recursos computacionais de forma eficiente para atender requisitos do sistema e manter essa eficiência conforme a demanda muda.

Práticas-chave:

- Democratizar tecnologias avançadas
- Globalizar em minutos
- Usar arquiteturas serverless
- Experimentar com mais frequência
- Mechanical sympathy

Otimização de Custos **PILAR 5**

Evitar gastos desnecessários e executar sistemas para entregar valor de negócio ao menor preço.

Práticas-chave:

- Adotar modelo de consumo
- Medir eficiência geral
- Parar de gastar com data centers
- Analisar e atribuir gastos
- Usar serviços gerenciados

Sustentabilidade **PILAR 6**

Minimizar os impactos ambientais da execução de workloads na nuvem.

Práticas-chave:

- Entender seu impacto
- Estabelecer metas de sustentabilidade
- Maximizar utilização
- Usar serviços gerenciados
- Reduzir impacto downstream

Perfil do Candidato

O exame é destinado a profissionais com experiência avançada em AWS:

Requisito	Descrição
Experiência	2+ anos de experiência prática projetando e implantando arquiteturas na AWS
Habilidade	Avaliar requisitos de aplicações e fazer recomendações arquiteturais para implementação e deploy
Conhecimento	Orientar a organização sobre melhores práticas de design multi-conta, multi-região e multi-tier
Escopo	Projetar soluções que atendam requisitos de segurança, confiabilidade, performance e custo
Diferencial	Capacidade de avaliar trade-offs entre diferentes abordagens arquiteturais

Tecnologias e Conceitos em Escopo

Compute & Containers **INFRAESTRUTURA**

- EC2 (Auto Scaling, Spot, Reserved)
- ECS / EKS / Fargate
- Lambda (serverless)
- Elastic Beanstalk
- App Runner
- Batch

Networking **CONECTIVIDADE**

- VPC, Subnets, Route Tables
- Transit Gateway, VPC Peering
- Direct Connect, Site-to-Site VPN
- PrivateLink, Global Accelerator
- Route 53, CloudFront
- Cloud WAN

Data & Analytics **DADOS**

- Data Lakes (S3 + Glue + Athena)
- Kinesis (Streams, Firehose, Analytics)
- Redshift, EMR
- DynamoDB, Aurora, ElastiCache
- Neptune, Timestream, QLDB
- QuickSight

DevOps & IaC **AUTOMAÇÃO**

- CI/CD (CodePipeline, CodeBuild, CodeDeploy)
- CloudFormation, CDK
- Systems Manager
- Config, CloudTrail
- EventBridge, Step Functions

Security & Governance

SEGURANÇA

- IAM, Organizations, SCPs
- Control Tower, Landing Zone
- GuardDuty, Security Hub, Inspector
- KMS, Secrets Manager, ACM
- WAF, Shield, Firewall Manager
- Macie, Detective

Migration & DR

MIGRAÇÃO

- 6 Rs de Migração
- Migration Hub, DMS, SCT
- DataSync, Transfer Family
- Snow Family
- DR Strategies (Pilot Light, Warm Standby)
- Elastic Disaster Recovery

Dicas Estratégicas para o Exame

Gestão de Tempo

180 MIN

- 75 questões em 180 minutos = ~2.4 min/questão
- Questões são longas (cenários complexos)
- Marque para revisão e siga em frente
- Não gaste mais de 3 min em uma questão
- Reserve 15 min para revisão final

Estratégia de Resposta **TÉCNICA**

- Leia os requisitos ANTES das opções
- Identifique palavras-chave no enunciado
- Elimine opções claramente erradas
- "Menor esforço operacional" = serverless/gerenciado
- "Custo-efetivo" = Reserved/Spot/lifecycle
- "Alta disponibilidade" = multi-AZ/multi-região

Padrões de Questão **RECONHECIMENTO**

- Cenários multi-conta → Organizations + SCPs
- Conectividade híbrida → Direct Connect + VPN
- DR com baixo RTO → Warm Standby/Multi-Site
- Migração de banco → DMS + SCT
- Dados em tempo real → Kinesis
- Desacoplamento → SQS/SNS/EventBridge

Armadilhas Comuns **ATENÇÃO**

- Opção correta tecnicamente mas não atende requisito
- Soluções over-engineered (complexas demais)
- Confundir "pode funcionar" com "melhor opção"
- Ignorar requisitos de compliance/região
- Não considerar custos operacionais
- Esquecer de ler TODAS as opções

 **Regra de ouro:** A resposta correta no SAP-C02 é sempre a que melhor equilibra os requisitos do cenário (custo, performance, segurança, operação) com o menor esforço operacional possível. AWS sempre prefere serviços gerenciados sobre soluções self-managed.

 SA Professional — Módulo 02

Complexidade Organizacional (26%)

Design de soluções para conectividade de rede, estratégia multi-conta, DNS híbrido, federação de identidade e governança em escala empresarial.

Conectividade de Rede

Serviços e padrões para conectar VPCs, data centers on-premises e escritórios remotos:

Transit Gateway

AWS Transit Gateway HUB CENTRAL

Hub de rede regional que conecta VPCs e redes on-premises através de um ponto central.

Características:

- Hub-and-spoke: conecta milhares de VPCs
- Suporta peering inter-região (Transit Gateway Peering)
- Route tables para segmentação de tráfego
- Suporta multicast
- Bandwidth: até 50 Gbps por attachment
- Integra com Direct Connect Gateway e VPN

Quando usar:

- 3+ VPCs que precisam se comunicar
- Topologia hub-and-spoke
- Segmentação de rede entre ambientes
- Conectividade transitiva entre VPCs

Transit Gateway vs VPC Peering

DECISÃO

VPC Peering:

- Conexão 1:1 entre duas VPCs
- Não transitivo ($A \leftrightarrow B$ e $B \leftrightarrow C \neq A \leftrightarrow C$)
- Sem custo de hora (só transferência de dados)
- Suporta inter-região
- Melhor para: poucas VPCs (2-3)

Transit Gateway:

- Hub central para N VPCs
- Transitivo por padrão
- Custo por hora + transferência
- Route tables para segmentação
- Melhor para: muitas VPCs, topologia complexa

Regra: <5 VPCs sem transitividade → Peering

5+ VPCs ou transitividade → Transit Gateway

Direct Connect

AWS Direct Connect **CONEXÃO DEDICADA**

Conexão de rede dedicada entre seu data center e a AWS. Não passa pela internet pública.

Tipos de conexão:

- Dedicated: 1 Gbps, 10 Gbps, 100 Gbps (porta física exclusiva)
- Hosted: 50 Mbps a 10 Gbps (via parceiro AWS)

Virtual Interfaces (VIFs):

- Private VIF → acesso a VPCs (via VGW ou Direct Connect Gateway)
- Public VIF → acesso a serviços públicos AWS (S3, DynamoDB)
- Transit VIF → acesso via Transit Gateway

Resiliência:

- Single connection = single point of failure
- High Resiliency: 2 conexões em 2 locais diferentes
- Maximum Resiliency: 2 conexões em cada um de 2 locais (4 total)

Lead time: semanas a meses para provisionar!

Direct Connect Gateway **MULTI-REGIÃO**

Permite acessar VPCs em múltiplas regiões através de uma única conexão Direct Connect.

Sem DX Gateway:

- 1 Direct Connect → 1 região apenas
- Precisa de conexão por região

Com DX Gateway:

- 1 Direct Connect → múltiplas regiões
- Conecta a VGWs ou Transit Gateways
- Global (não é regional)

Combinação com Transit Gateway:

- DX → DX Gateway → Transit Gateway → VPCs
- Acesso a centenas de VPCs via 1 conexão
- Transit VIF necessário

VPN

Site-to-Site VPN IPSEC

Conexão criptografada (IPsec) entre seu data center e a AWS via internet pública.

Componentes:

- Virtual Private Gateway (VGW) – lado AWS
- Customer Gateway (CGW) – lado on-premises
- 2 túneis IPsec para redundância

Características:

- Setup em minutos (vs semanas do DX)
- Bandwidth: até ~1.25 Gbps por túnel
- Criptografia IPsec
- Pode usar como backup do Direct Connect

VPN + Direct Connect (backup):

- DX como primário (baixa latência)
- VPN como failover (via internet)
- BGP para failover automático

Client VPN **USUÁRIOS REMOTOS**

VPN gerenciada para conectar usuários individuais (laptops) à AWS ou rede corporativa.

Características:

- Baseado em OpenVPN
- Escala elástica (milhares de usuários)
- Autenticação: AD, SAML, certificados
- Split-tunnel ou full-tunnel
- Integra com Security Groups

Quando usar:

- Funcionários remotos acessando VPCs
- Acesso seguro a recursos internos
- Substituir VPN on-premises

Diferença do Site-to-Site:

- Site-to-Site = rede-a-rede (data center → AWS)
- Client VPN = usuário-a-rede (laptop → AWS)

Outros Serviços de Rede

AWS PrivateLink ENDPOINT SERVICES

Acesso privado a serviços AWS ou serviços de terceiros sem expor tráfego à internet.

Tipos de VPC Endpoints:

- Interface Endpoint (PrivateLink): ENI na sua VPC
 - Maioria dos serviços AWS
 - Serviços de terceiros no Marketplace
 - Seus próprios serviços (via NLB)
- Gateway Endpoint: entrada na route table
 - Apenas S3 e DynamoDB
 - Sem custo adicional

Quando usar PrivateLink:

- Expor serviço para outras contas sem peering
- Acesso privado a SaaS
- Compliance: tráfego não pode sair da rede AWS

Global Accelerator PERFORMANCE

Melhora performance de aplicações globais usando a rede backbone da AWS (Anycast IPs).

Como funciona:

- 2 IPs Anycast estáticos globais
- Tráfego entra na edge location mais próxima
- Viaja pela rede privada AWS (não internet)
- Roteamento inteligente para endpoints saudáveis

vs CloudFront:

- CloudFront: cache de conteúdo estático/dinâmico
- Global Accelerator: TCP/UDP sem cache
 - Gaming, IoT, VoIP
 - Failover instantâneo entre regiões
 - IPs estáticos (whitelist em firewalls)

AWS Cloud WAN REDE GLOBAL

Serviço gerenciado para construir, gerenciar e monitorar uma rede global unificada.

Características:

- Rede global como serviço
- Política centralizada (network policy)
- Segmentação automática
- Integra VPCs, DX, VPN, SD-WAN
- Dashboard de monitoramento

Quando usar:

- Rede global complexa (muitas regiões)
- Substituir Transit Gateway multi-região
- Gerenciamento centralizado de rede
- Segmentação por política

Estratégia Multi-Conta

AWS Organizations **GOVERNANÇA**

Gerenciamento centralizado de múltiplas contas AWS com billing consolidado e políticas.

Estrutura:

- Management Account (payer) – raiz
- Organizational Units (OUs) – agrupamento lógico
- Member Accounts – contas individuais

Service Control Policies (SCPs):

- Definem permissões MÁXIMAS para contas/OUs
- NÃO concedem permissões (apenas restringem)
- Não afetam a Management Account
- Herança hierárquica (OU pai → OU filho)

Exemplo de SCP:

- Deny: criar recursos fora de regiões permitidas
- Deny: desabilitar CloudTrail
- Deny: sair da organização

AWS Control Tower **LANDING ZONE**

Configura e governa um ambiente multi-conta seguro seguindo melhores práticas (Landing Zone).

Componentes:

- Landing Zone: ambiente multi-conta pré-configurado
- Guardrails: regras preventivas e detectivas
 - Preventive: SCPs (bloqueia ações)
 - Detective: Config Rules (detecta violações)
- Account Factory: criação padronizada de contas
- Dashboard: visão de compliance

Contas criadas automaticamente:

- Log Archive Account (CloudTrail centralizado)
- Audit Account (acesso cross-account para auditoria)

Quando usar:

- Novo ambiente AWS multi-conta
- Padronizar criação de contas
- Governança automatizada

Padrão de Contas Recomendado

Conta/OU	Propósito	Serviços Principais
Management Account	Billing, Organizations, SCPs	Organizations, Billing, IAM Identity Center
Security OU	Auditoria e logs centralizados	CloudTrail, Config, GuardDuty (delegated admin)
Infrastructure OU	Serviços compartilhados de rede	Transit Gateway, Direct Connect, DNS
Sandbox OU	Experimentação com limites de custo	Budgets, SCPs restritivas
Workloads OU	Aplicações (dev/staging/prod)	Separação por ambiente e aplicação

DNS Híbrido

Route 53 Resolver DNS HÍBRIDO

Resolve consultas DNS entre a AWS e redes on-premises.

Endpoints:

- Inbound Endpoint:
 - On-premises DNS → AWS (resolve domínios privados AWS)
 - DNS on-prem encaminha queries para o endpoint
 - ENIs na VPC que recebem queries
- Outbound Endpoint:
 - AWS → On-premises DNS (resolve domínios corporativos)
 - Route 53 encaminha queries via regras
 - Forwarding Rules definem quais domínios

Resolver Rules:

- Forward: encaminha domínio X para DNS Y
- System: usa resolução padrão do Route 53
- Compartilháveis via RAM (Resource Access Manager)

Cenário típico:

- corp.internal → Outbound → DNS on-premises
- aws.internal → Inbound → Route 53 Private Hosted Zone

Private Hosted Zones DNS PRIVADO

Zonas DNS que respondem apenas dentro de VPCs associadas.

Características:

- Resolução apenas dentro de VPCs associadas
- Pode associar VPCs de outras contas (cross-account)
- Split-view DNS: mesmo domínio, respostas diferentes
 - Público: api.empresa.com → IP público
 - Privado: api.empresa.com → IP privado

Cross-account:

- Conta A cria a Private Hosted Zone
- Conta A autoriza associação da VPC da Conta B
- Conta B aceita e associa sua VPC
- Ambas resolvem os mesmos registros

Federação de Identidade

IAM Identity Center **sso**

Gerenciamento centralizado de acesso SSO para múltiplas contas AWS e aplicações.

Anteriormente: AWS SSO

Características:

- Login único para todas as contas da organização
- Integra com IdPs externos (AD, Okta, Azure AD)
- Permission Sets: definem acesso por conta
- Atribuição: usuário/grupo → permission set → conta

Identity Sources:

- Identity Center directory (built-in)
- Active Directory (AD Connector ou AWS Managed AD)
- External IdP (SAML 2.0 – Okta, Azure AD, etc.)

Quando usar:

- Acesso humano a múltiplas contas AWS
- SSO corporativo
- Gerenciamento centralizado de permissões

SAML 2.0 / OIDC FEDERAÇÃO

Protocolos para federação de identidade com provedores externos.

SAML 2.0:

- Padrão enterprise (Active Directory, ADFS)
- Assume Role via STS (AssumeRoleWithSAML)
- Console AWS ou API/CLI
- Fluxo: User → IdP → SAML Assertion → STS → Credentials

OIDC (OpenID Connect):

- Padrão web/mobile (Google, Facebook, Cognito)
- AssumeRoleWithWebIdentity
- Melhor usar Cognito como intermediário

Amazon Cognito:

- User Pools: autenticação (sign-up/sign-in)
- Identity Pools: autorização (credenciais AWS temporárias)
- Federação com SAML, OIDC, social IdPs
- Quando usar: aplicações web/mobile com usuários externos

Governança e Compliance

AWS Config COMPLIANCE

Avalia, audita e monitora configurações de recursos AWS continuamente.

Componentes:

- Config Rules: regras de compliance
 - Managed Rules (pré-definidas pela AWS)
 - Custom Rules (Lambda)
- Conformance Packs: conjunto de regras
- Aggregator: visão multi-conta/multi-região
- Remediation: correção automática (SSM Automation)

Exemplos de regras:

- EBS volumes devem ser criptografados
- Security Groups não podem ter 0.0.0.0/0
- S3 buckets devem ter versionamento
- RDS deve ter backup habilitado

Multi-conta:

- Config Aggregator na conta de auditoria
- Delegated Administrator
- Organization Config Rules

CloudTrail **AUDITORIA**

Registra todas as chamadas de API feitas na conta AWS (quem fez o quê, quando).

Tipos de eventos:

- Management Events: operações de controle
(CreateBucket, AttachPolicy, RunInstances)
- Data Events: operações em dados
(GetObject, PutObject, Invoke Lambda)
- Insights Events: atividade incomum

Organization Trail:

- Trail na Management Account
- Captura eventos de TODAS as contas
- Centraliza logs em S3 da conta de segurança
- Imutável com S3 Object Lock

Integração:

- S3 (armazenamento de longo prazo)
- CloudWatch Logs (alertas em tempo real)
- Athena (queries SQL nos logs)
- EventBridge (automação baseada em eventos)

GuardDuty DETECÇÃO DE AMEAÇAS

Detecção inteligente de ameaças usando ML, análise de anomalias e threat intelligence.

Fontes de dados:

- CloudTrail Events (Management + Data)
- VPC Flow Logs
- DNS Logs
- EKS Audit Logs
- S3 Data Events

Tipos de findings:

- Reconnaissance (port scanning, API probing)
- Instance Compromise (crypto mining, C&C)
- Account Compromise (credenciais vazadas)
- S3 Compromise (acesso anômalo)

Multi-conta:

- Delegated Administrator na conta de segurança
- Habilitar para todas as contas da organização
- Findings centralizados
- Auto-enable para novas contas

Security Hub POSTURA

Visão centralizada de segurança e compliance. Agrega findings de múltiplos serviços.

Agrega findings de:

- GuardDuty, Inspector, Macie
- Firewall Manager, IAM Access Analyzer
- Config, terceiros (Qualys, Palo Alto, etc.)

Security Standards:

- AWS Foundational Security Best Practices
- CIS AWS Foundations Benchmark
- PCI DSS
- NIST 800-53

Automação:

- Custom Actions → EventBridge → Lambda
- Automated response (quarantine, notify)
- Cross-account aggregation
- Integration com SIEM (Splunk, etc.)

 **Para o exame:** "Detectar ameaças" → GuardDuty | "Compliance de configuração" → Config | "Auditoria de API" → CloudTrail | "Visão centralizada de segurança" → Security Hub | "Governança multi-conta" → Organizations + SCPs + Control Tower

 SA Professional — Módulo 03

Design para Novas Soluções (29%)

Projetar arquiteturas para novos workloads: compute, storage, databases, integração de aplicações, analytics e estratégias de Disaster Recovery.

Compute

AWS Lambda SERVERLESS

Execução de código sem gerenciar servidores. Escala automaticamente de 0 a milhares de execuções concorrentes.

Limites importantes:

- Timeout: máximo 15 minutos
- Memória: 128 MB a 10 GB
- Payload: 6 MB (síncrono), 256 KB (assíncrono)
- Concorrência: 1000 (padrão, pode aumentar)
- /tmp storage: até 10 GB
- Deployment package: 50 MB (zip), 250 MB (descompactado)

Invocação:

- Síncrona: API Gateway, ALB, SDK
- Assíncrona: S3, SNS, EventBridge
- Event Source Mapping: SQS, Kinesis, DynamoDB Streams

Quando usar:

- Processamento event-driven
- APIs com tráfego variável
- Processamento de dados em tempo real
- Automação de tarefas

ECS / EKS / Fargate CONTAINERS

Orquestração de containers na AWS com diferentes níveis de controle.

ECS (Elastic Container Service):

- Orquestrador proprietário AWS
- Mais simples que Kubernetes
- Integração nativa com ALB, IAM, CloudWatch
- Launch types: EC2 ou Fargate

EKS (Elastic Kubernetes Service):

- Kubernetes gerenciado
- Compatível com ecossistema K8s
- Quando: equipe já usa K8s, multi-cloud
- Launch types: EC2, Fargate, ou Outposts

Fargate:

- Serverless para containers
- Sem gerenciar EC2 instances
- Paga por vCPU + memória usada
- Quando: não quer gerenciar infraestrutura

Decisão:

- Simples + AWS-native → ECS + Fargate
- K8s expertise + portabilidade → EKS
- Controle total de instâncias → ECS/EKS + EC2

EC2 Auto Scaling ELASTICIDADE

Escala automática de instâncias EC2 baseada em métricas ou schedules.

Tipos de scaling:

- Target Tracking: mantém métrica no target
(ex: CPU em 50%)
- Step Scaling: ações baseadas em thresholds
- Scheduled: horários pré-definidos
- Predictive: ML prevê demanda futura

Políticas avançadas:

- Warm Pools: instâncias pré-aquecidas
- Instance Refresh: rolling update do ASG
- Mixed Instances: On-Demand + Spot
- Capacity Rebalancing: substitui Spot antes de interrupção

Launch Template vs Launch Configuration:

- Launch Template: versioning, múltiplos instance types
- Launch Configuration: legado, sem versioning
- Sempre use Launch Template!

EC2 Spot Instances ECONOMIA

Capacidade EC2 não utilizada com até 90% de desconto. Pode ser interrompida com 2 min de aviso.

Estratégias:

- Spot Fleet: pool de Spot + On-Demand
- Diversificação: múltiplos instance types e AZs
- Capacity Optimized: escolhe pool com mais capacidade
- Lowest Price: escolhe mais barato (risco maior)

Quando usar:

- Workloads tolerantes a interrupção
- Batch processing, CI/CD, rendering
- Big data (EMR, Spark)
- Containers com ECS/EKS

Quando NÃO usar:

- Databases
- Aplicações stateful críticas
- Workloads que não podem ser interrompidos

Storage

S3 Storage Classes **OBJETO**

Classes de armazenamento com diferentes trade-offs de custo, disponibilidade e acesso.

Classes (custo crescente de armazenamento → acesso):

S3 Standard	Acesso frequente, 3+ AZs	
S3 Intelligent-Tier	Automático por padrão	
S3 Standard-IA	Infrequente, 3+ AZs	
S3 One Zone-IA	Infrequente, 1 AZ	
S3 Glacier Instant	Arquivo, ms de acesso	
S3 Glacier Flexible	Arquivo, min-hrs	
S3 Glacier Deep Arch	Arquivo, 12-48 hrs	

Lifecycle Rules:

- Transition: mover entre classes automaticamente
- Expiration: deletar objetos após N dias
- Pode combinar com versionamento

S3 Intelligent-Tiering:

- Move automaticamente entre tiers
- Sem taxa de retrieval
- Ideal quando padrão de acesso é desconhecido

EFS / FSx **FILE SYSTEM**

Sistemas de arquivos gerenciados para diferentes workloads.

Amazon EFS:

- NFS gerenciado (Linux)
- Multi-AZ por padrão
- Escala automaticamente (petabytes)
- Performance modes: General Purpose, Max I/O
- Throughput modes: Bursting, Provisioned, Elastic
- Quando: compartilhar arquivos entre EC2/containers Linux

Amazon FSx for Windows:

- SMB + NTFS gerenciado
- Integra com Active Directory
- Quando: workloads Windows, SQL Server

Amazon FSx for Lustre:

- File system de alta performance
- Integra com S3 (hot/cold data)
- Quando: HPC, ML training, rendering

Amazon FSx for NetApp ONTAP:

- Multi-protocolo (NFS, SMB, iSCSI)
- Quando: migrar NetApp on-premises

Storage Gateway **HÍBRIDO**

Ponte entre armazenamento on-premises e cloud storage (S3, FSx, EBS).

Tipos:

- S3 File Gateway:
 - NFS/SMB → S3 (com cache local)
 - Quando: backup de arquivos para S3
- FSx File Gateway:
 - SMB → FSx for Windows (com cache local)
 - Quando: acesso local a FSx
- Volume Gateway:
 - iSCSI → EBS Snapshots em S3
 - Cached: dados em S3, cache local
 - Stored: dados locais, backup em S3
- Tape Gateway:
 - iSCSI VTL → S3 Glacier
 - Quando: substituir fitas físicas de backup

Database

Amazon Aurora RELACIONAL

Banco relacional gerenciado compatível com MySQL e PostgreSQL, até 5x mais rápido.

Características:

- Storage auto-scaling (até 128 TB)
- 6 cópias dos dados em 3 AZs
- Failover automático (<30 segundos)
- Até 15 read replicas (baixa latência)

Aurora Global Database:

- Replicação cross-region (<1 segundo lag)
- Failover cross-region (RPO ~1s, RTO ~1min)
- Até 5 regiões secundárias

Aurora Serverless v2:

- Escala automaticamente (0.5 a 128 ACUs)
- Paga por uso (sem instâncias ociosas)
- Quando: tráfego imprevisível, dev/test

Aurora Multi-Master:

- Múltiplos nós de escrita
- Alta disponibilidade de escrita
- Quando: zero downtime para escritas

DynamoDB NOSQL

Banco NoSQL serverless com latência de milissegundos em qualquer escala.

Características:

- Key-value e document store
- Single-digit millisecond latency
- Auto-scaling ou on-demand capacity
- Global Tables: multi-região active-active
- DynamoDB Streams: CDC (Change Data Capture)
- TTL: expiração automática de itens

Capacidade:

- Provisioned: RCU/WCU definidos (mais barato se previsível)
- On-Demand: paga por request (imprevisível)

DAX (DynamoDB Accelerator):

- Cache in-memory para DynamoDB
- Microsegundos de latência
- Compatível com API DynamoDB (drop-in)

Quando usar:

- Alta escala, baixa latência
- Schema flexível
- Gaming, IoT, session store
- Não usar: queries complexas, joins, transações ACID complexas

ElastiCache **CACHE**

Cache in-memory gerenciado (Redis ou Memcached) para reduzir latência de leitura.

Redis vs Memcached:

Feature	Redis	Memcached
Estruturas	Strings, Lists, Sets, Hashes, Sorted Sets	Strings apenas
Replicação	Sim (Multi-AZ)	Não
Persistência	Sim (AOF, RDB)	Não
Cluster	Sim (sharding)	Sim
Pub/Sub	Sim	Não
Lua Scripts	Sim	Não

Padrões de caching:

- Lazy Loading: carrega no cache apenas quando solicitado
- Write-Through: escreve no cache junto com o DB
- TTL: expiração para evitar dados stale

Quando usar Redis:

- Session store, leaderboards, real-time analytics
- Pub/Sub, geospatial, rate limiting

Bancos Especializados

PURPOSE-BUILT

Amazon Neptune:

- Graph database (relações complexas)
- Social networks, fraud detection, knowledge graphs
- Suporta Gremlin e SPARQL

Amazon Timestream:

- Time-series database
- IoT, DevOps metrics, industrial telemetry
- 1000x mais rápido que bancos relacionais para time-series

Amazon QLDB:

- Ledger database (imutável, verificável)
- Histórico completo de mudanças (cryptographic verification)
- Supply chain, financial transactions, registros regulatórios

Amazon Keyspaces:

- Cassandra-compatible (wide-column)
- Quando: migrar workloads Cassandra

Amazon DocumentDB:

- MongoDB-compatible
- Quando: migrar workloads MongoDB

Integração de Aplicações

Amazon SQS QUEUE

Fila de mensagens totalmente gerenciada para desacoplamento de componentes.

Tipos:

- Standard: throughput ilimitado, at-least-once, sem ordem
- FIFO: 300 msg/s (3000 com batching), exactly-once, ordenado

Características:

- Retention: 1 min a 14 dias (default 4 dias)
- Message size: até 256 KB
- Visibility Timeout: tempo que msg fica invisível após leitura
- Dead Letter Queue (DLQ): msgs que falharam N vezes
- Long Polling: reduz chamadas vazias (WaitTimeSeconds)

Padrões:

- Fan-out: SNS → múltiplas SQS queues
- Request buffering: absorver picos de tráfego
- Decoupling: separar producer de consumer

Amazon SNS **PUB/SUB**

Serviço de pub/sub para enviar mensagens a múltiplos subscribers simultaneamente.

Subscribers:

- SQS, Lambda, HTTP/S, Email, SMS
- Kinesis Data Firehose
- Mobile push notifications

Características:

- Fan-out: 1 mensagem → N subscribers
- Message filtering: filtrar por atributos
- FIFO topics: ordenação e deduplicação
- Cross-account subscriptions

Padrão SNS + SQS Fan-out:

- SNS Topic publica mensagem
- Múltiplas SQS queues inscrevem
- Cada queue processa independentemente
- Desacoplamento total entre consumers

Amazon EventBridge EVENT BUS

Barramento de eventos serverless para construir arquiteturas event-driven.

Características:

- Event Bus: recebe eventos de múltiplas fontes
- Rules: filtram e roteiam eventos para targets
- Schema Registry: descoberta automática de schemas
- Archive & Replay: re-processar eventos passados
- Cross-account / cross-region delivery

Fontes de eventos:

- Serviços AWS (100+)
- SaaS partners (Zendesk, Datadog, Auth0)
- Custom applications

vs SNS:

- EventBridge: content-based filtering avançado, schema discovery, SaaS integration, archive/replay
- SNS: fan-out simples, alta throughput, mobile push

Quando usar EventBridge:

- Arquitetura event-driven complexa
- Integração com SaaS
- Roteamento baseado em conteúdo do evento

Step Functions / AppSync ORQUESTRAÇÃO

AWS Step Functions:

- Orquestração visual de workflows
- State machines (JSON/YAML)
- Integra com 200+ serviços AWS
- Standard: longa duração (até 1 ano), exactly-once
- Express: alta throughput, curta duração (5 min)
- Quando: coordenar múltiplos Lambda/serviços

AWS AppSync:

- GraphQL gerenciado
- Real-time subscriptions (WebSocket)
- Offline sync (mobile)
- Resolvers: DynamoDB, Lambda, HTTP, RDS
- Quando: APIs GraphQL, real-time, mobile offline

Amazon MQ:

- Message broker gerenciado (ActiveMQ, RabbitMQ)
- Quando: migrar de brokers on-premises
- Suporta protocolos: AMQP, MQTT, STOMP, OpenWire
- NÃO é serverless (instâncias gerenciadas)

Analytics

Amazon Kinesis **STREAMING**

Plataforma para processamento de dados em streaming em tempo real.

Kinesis Data Streams:

- Ingestão de dados em tempo real
- Shards: unidade de throughput (1 MB/s in, 2 MB/s out)
- Retention: 24h (default) até 365 dias
- Consumers: Lambda, KCL, Kinesis Analytics
- Quando: processamento customizado em real-time

Kinesis Data Firehose:

- Entrega de dados para destinos (near real-time)
- Destinos: S3, Redshift, OpenSearch, Splunk, HTTP
- Transformação com Lambda
- Buffer: 60s-900s ou 1-128 MB
- Quando: ETL simples para S3/Redshift

Kinesis Data Analytics:

- SQL ou Apache Flink em streaming data
- Quando: analytics em tempo real

vs SQS:

- Kinesis: múltiplos consumers, replay, ordering
- SQS: desacoplamento simples, delete após processar

Athena / Redshift**QUERY****Amazon Athena:**

- Query SQL serverless sobre S3
- Paga por dados escaneados
- Formatos: Parquet, ORC, JSON, CSV
- Integra com Glue Data Catalog
- Quando: queries ad-hoc, data lake analytics
- Otimização: Parquet/ORC + particionamento

Amazon Redshift:

- Data warehouse petabyte-scale
- Columnar storage, MPP (Massively Parallel)
- Redshift Spectrum: query S3 sem carregar dados
- Redshift Serverless: sem gerenciar clusters
- Quando: BI, reporting complexo, joins pesados

Decisão:

- Queries ad-hoc, serverless → Athena
- BI complexo, dashboards, joins → Redshift
- Ambos podem consultar S3 (data lake)

EMR / Glue **ETL**

Amazon EMR:

- Hadoop/Spark gerenciado
- Frameworks: Spark, Hive, Presto, HBase, Flink
- EC2, EKS, ou Serverless
- Quando: big data processing, ML em escala

AWS Glue:

- ETL serverless
- Glue Data Catalog: metadados centralizados
- Glue Crawlers: descoberta automática de schema
- Glue Jobs: Spark ETL (Python/Scala)
- Glue Studio: visual ETL
- Quando: ETL para data lake, catalogação

Amazon QuickSight:

- BI serverless (dashboards e visualizações)
- SPICE: in-memory engine
- ML Insights: anomaly detection, forecasting
- Embedding: dashboards em aplicações
- Quando: visualização de dados, reporting

Estratégias de Disaster Recovery

Estratégias ordenadas por custo (menor → maior) e RTO/RPO (maior → menor):

Estratégia	RTO	RPO	Custo	Descrição
Backup & Restore	Horas	Horas	\$	Backups em S3/outra região. Restaura quando necessário. Sem infraestrutura ativa na DR.
Pilot Light	10s min	Minutos	\$\$	Core mínimo rodando (DB replicado). Compute desligado, liga sob demanda.
Warm Standby	Minutos	Segundos	\$\$\$	Versão reduzida do ambiente completo rodando. Scale up quando ativado.
Multi-Site Active-Active	~0 (real-time)	~0	\$\$\$\$	Ambiente completo em 2+ regiões. Tráfego distribuído. Failover instantâneo.

Backup & Restore \$

Implementação:

- AWS Backup: backups automatizados cross-region
- S3 Cross-Region Replication
- RDS/Aurora snapshots copiados para outra região
- AMIs copiadas para outra região

Recuperação:

- Restaurar backups na região DR
- Provisionar infraestrutura (CloudFormation)
- Atualizar DNS

Quando usar:

- Workloads não-críticos
- Budget limitado
- RTO de horas é aceitável

Pilot Light \$\$

Implementação:

- DB replicado continuamente (Aurora Global, DynamoDB Global)
- Infraestrutura definida em IaC (pronta para deploy)
- Compute DESLIGADO (AMIs prontas)
- Route 53 health checks

Ativação:

- Promover DB replica para primary
- Iniciar instâncias EC2 / scale up ASG
- Atualizar DNS (Route 53 failover)

Quando usar:

- Core business com RTO de ~10 minutos
- Custo moderado aceitável
- DB é o componente mais crítico

Warm Standby \$\$\$

Implementação:

- Ambiente completo rodando em escala REDUZIDA
- DB: Aurora Global Database (read replica)
- Compute: ASG com min capacity baixo
- Load Balancer ativo
- Route 53 weighted/failover routing

Ativação:

- Scale up ASG (aumentar min/desired)
- Promover DB replica
- Shift DNS traffic (Route 53)

Quando usar:

- Aplicações business-critical
- RTO de minutos
- Pode justificar custo de infra reduzida 24/7

Multi-Site Active-Active \$\$\$\$

Implementação:

- Ambiente COMPLETO em 2+ regiões
- Tráfego distribuído (Route 53 latency/weighted)
- DynamoDB Global Tables (multi-region writes)
- Aurora Global Database (multi-master)
- CloudFront com múltiplas origens

Failover:

- Automático via Route 53 health checks
- Sem intervenção manual
- Zero downtime

Quando usar:

- Aplicações mission-critical
- Zero tolerance para downtime
- Compliance com requisitos de DR rigorosos
- Budget não é restrição principal

 **Para o exame:** Identifique RTO/RPO no enunciado → escolha a estratégia adequada. "Menor custo com RTO de horas" → Backup & Restore. "RTO próximo de zero" → Multi-Site. "Custo-efetivo com RTO de minutos" → Warm Standby. AWS Elastic Disaster Recovery (DRS) automatiza Pilot Light/Warm Standby.



SA Professional — Módulo 04

Melhoria Contínua (25%)

Otimização de custos, performance, excelência operacional e confiabilidade de soluções existentes na AWS.

Otimização de Custos

Modelos de Compra EC2 PRICING

Escolher o modelo correto de compra pode reduzir custos em até 72%.

On-Demand:

- Paga por hora/segundo
- Sem compromisso
- Quando: workloads imprevisíveis, curta duração

Reserved Instances (RI):

- 1 ou 3 anos de compromisso
- Até 72% de desconto
- Standard RI: tipo fixo, pode vender no Marketplace
- Convertible RI: pode trocar tipo (menor desconto)
- Quando: workloads estáveis e previsíveis

Savings Plans:

- Compromisso de \$/hora por 1 ou 3 anos
- Compute SP: qualquer região, família, OS, tenancy
- EC2 Instance SP: família específica em uma região
- Mais flexível que RI
- Quando: preferir flexibilidade sobre RI

Spot Instances:

- Até 90% de desconto
- Pode ser interrompido (2 min aviso)
- Quando: batch, CI/CD, tolerante a falha

Dedicated Hosts:

- Servidor físico dedicado
- Quando: compliance, licenças por socket/core

Right-Sizing & Lifecycle OTIMIZAÇÃO

AWS Compute Optimizer:

- Analisa utilização de EC2, EBS, Lambda, ECS
- Recomenda instance types ideais
- Baseado em ML (14 dias de métricas)
- Identifica over-provisioned e under-provisioned

Right-Sizing:

- Reduzir instance type se CPU/memória <40%
- Considerar Graviton (ARM) – até 40% melhor preço/performance
- Usar T instances (burstable) para workloads variáveis

S3 Lifecycle Policies:

- Standard → Standard-IA (30+ dias)
- Standard-IA → Glacier Instant (90+ dias)
- Glacier → Deep Archive (180+ dias)
- Expiration: deletar após N dias
- S3 Intelligent-Tiering: automático

Outras otimizações:

- EBS: gp3 vs gp2 (gp3 é mais barato e configurável)
- NAT Gateway: considerar VPC endpoints para S3/DynamoDB
- Data transfer: usar CloudFront, VPC endpoints, mesma AZ
- Idle resources: ELBs sem targets, EIPs não associados

Otimização de Performance

CloudFront **CDN**

CDN global que cacheia conteúdo em edge locations para reduzir latência.

Otimizações:

- Cache Policy: controle granular de cache keys
- Origin Request Policy: headers enviados à origin
- Cache behaviors: diferentes configs por path pattern
- Lambda@Edge / CloudFront Functions:
 - Manipular request/response na edge
 - A/B testing, URL rewrite, auth

Performance:

- Compressão automática (gzip, Brotli)
- HTTP/2 e HTTP/3 (QUIC)
- Origin Shield: camada extra de cache
- Regional Edge Caches: entre edge e origin

Segurança:

- OAC (Origin Access Control) para S3
- WAF integration
- Geo-restriction
- Field-level encryption
- Signed URLs / Signed Cookies

Caching Strategies

LATÊNCIA

ElastiCache (Redis/Memcached):

- Cache de queries de banco de dados
- Session store
- Leaderboards, rate limiting

DAX (DynamoDB Accelerator):

- Cache transparente para DynamoDB
- Microsegundos de latência
- API-compatible (drop-in replacement)

API Gateway Caching:

- Cache de respostas da API
- TTL configurável (300s default)
- Invalidação por header ou flush

CloudFront Caching:

- Conteúdo estático e dinâmico
- TTL por behavior
- Invalidation para forçar refresh

Padrão de decisão:

- DB reads pesados → ElastiCache/DAX
- API responses → API Gateway cache
- Assets/páginas → CloudFront
- Session data → ElastiCache Redis

Read Replicas & Connection Pooling

DATABASE

Read Replicas:

- RDS: até 15 replicas (Aurora), 5 (outros engines)
- Offload de leitura do primary
- Cross-region replicas para DR e latência
- Promoção para standalone em caso de falha

Aurora Reader Endpoint:

- Load balancing automático entre replicas
- Connection string única para todas as replicas

RDS Proxy:

- Connection pooling gerenciado
- Reduz conexões ao banco (Lambda → RDS)
- Failover mais rápido (66% menos tempo)
- IAM authentication
- Quando: Lambda + RDS (evita connection exhaustion)

Global Accelerator para DB:

- Não aplicável diretamente
- Use Aurora Global Database para multi-região
- Route 53 latency-based para read replicas regionais

Excelência Operacional

Infrastructure as Code IAC

AWS CloudFormation:

- Templates declarativos (YAML/JSON)
- Stacks: unidade de deploy
- StackSets: deploy multi-conta/multi-região
- Drift Detection: detecta mudanças manuais
- Change Sets: preview de mudanças

AWS CDK (Cloud Development Kit):

- IaC em linguagens de programação
- TypeScript, Python, Java, C#, Go
- Compila para CloudFormation
- Constructs: abstrações reutilizáveis
- Quando: equipes de desenvolvimento preferem código

Terraform:

- Multi-cloud, HCL
- State management
- Quando: multi-cloud ou preferência da equipe

Melhores práticas:

- Versionamento em Git
- CI/CD para infraestrutura
- Ambientes separados (dev/staging/prod)
- Módulos reutilizáveis
- Drift detection periódico

Systems Manager

OPERAÇÕES

Componentes principais:

- Session Manager: acesso SSH sem abrir porta 22
- Patch Manager: patching automatizado
- Parameter Store: configurações e secrets
- Run Command: executar comandos em frota
- Automation: runbooks para tarefas operacionais
- State Manager: configuração desejada
- Inventory: inventário de software/hardware

Automation Runbooks:

- Documentos que definem ações automatizadas
- Aprovação manual opcional
- Integra com EventBridge para triggers
- Exemplos: restart instance, create AMI, remediate Config

OpsCenter:

- Visão centralizada de issues operacionais
- OpsItems: tickets de operação
- Integra com CloudWatch, Config, CloudFormation
- Runbooks para resolução

Config Rules & Auto-Remediation COMPLIANCE

AWS Config Rules:

- Avaliam compliance de recursos continuamente
- Managed Rules: 300+ regras pré-definidas
- Custom Rules: Lambda functions
- Organization Rules: aplicar em todas as contas

Auto-Remediation:

- Config Rule detecta non-compliance
- Trigger automático de SSM Automation
- Exemplos:
 - SG aberto → fechar porta
 - S3 sem encryption → habilitar
 - EBS sem backup → criar snapshot
 - Instance sem tag → adicionar tag

Conformance Packs:

- Conjunto de Config Rules + Remediations
- Templates pré-definidos (PCI-DSS, HIPAA, etc.)
- Deploy via Organizations (multi-conta)

Fluxo:

Config Rule (detecta) → SSM Automation (remedia)
→ EventBridge (notifica)
→ Security Hub (agrega)

Confiabilidade

Multi-AZ & Multi-Region HA

Multi-AZ (Alta Disponibilidade):

- RDS Multi-AZ: failover automático (standby síncrono)
- Aurora: 6 cópias em 3 AZs, failover <30s
- ELB: distribui tráfego entre AZs
- ASG: instâncias em múltiplas AZs
- S3: replicação automática em 3+ AZs
- EFS: multi-AZ por padrão

Multi-Region (DR + Latência):

- Aurora Global Database: <1s replication lag
- DynamoDB Global Tables: active-active multi-region
- S3 Cross-Region Replication (CRR)
- CloudFront: cache global
- Route 53: latency-based / failover routing
- API Gateway: regional endpoints por região

Decisão:

- HA dentro de uma região → Multi-AZ
- DR ou latência global → Multi-Region
- Ambos para máxima resiliência

Route 53 Failover DNS

Routing Policies:

- Simple: um registro, um destino
- Weighted: distribuir % entre destinos
- Latency: rota para região mais próxima
- Failover: primary/secondary com health check
- Geolocation: rota por localização do usuário
- Geoproximity: rota por proximidade (com bias)
- Multivalue: múltiplos IPs com health check

Health Checks:

- Endpoint: HTTP/HTTPS/TCP para IP ou domínio
- Calculated: combina múltiplos health checks (AND/OR)
- CloudWatch Alarm: baseado em métrica

Failover Pattern:

- Primary: região principal (health check ativo)
- Secondary: região DR (ativado quando primary falha)
- Failover automático em ~30-60 segundos
- Pode combinar com Global Accelerator

Chaos Engineering

RESILIÊNCIA

AWS Fault Injection Simulator (FIS):

- Serviço gerenciado para chaos engineering
- Injeta falhas controladas em recursos AWS
- Experiment templates: definem ações e targets
- Stop conditions: limites de segurança

Tipos de experimentos:

- EC2: terminate instances, stress CPU/memory
- ECS: stop tasks, drain container instances
- EKS: terminate pods, inject network latency
- RDS: failover, reboot
- Network: packet loss, latency injection

Melhores práticas:

- Comece em ambientes não-produtivos
- Defina hipóteses antes do experimento
- Monitore métricas durante o teste
- Stop conditions para limitar blast radius
- Game Days: exercícios regulares de DR

Objetivo:

- Validar que sistemas se recuperam de falhas
- Encontrar pontos fracos antes que causem outages
- Construir confiança na resiliência da arquitetura

 **Para o exame:** "Otimizar custos" → Reserved/Savings Plans + right-sizing + lifecycle policies. "Melhorar performance" → caching (CloudFront, ElastiCache, DAX) + read replicas. "Reduzir esforço operacional" → serviços gerenciados + automação (SSM, Config auto-remediation). "Aumentar resiliência" → multi-AZ + health checks + chaos engineering.

Migração e Modernização (20%)

Estratégias de migração (6 Rs), serviços de migração, transferência de dados e modernização de workloads para a nuvem AWS.

6 Rs de Migração

Estratégias para migrar workloads para a nuvem, ordenadas por complexidade crescente:

Estratégia	Descrição	Esforço	Quando Usar
Rehost (Lift & Shift)	Mover para a nuvem sem modificações. Mesmo OS, mesmo app.	Baixo	Migração rápida, deadline apertado, apps legados
Replatform (Lift & Reshape)	Pequenas otimizações sem mudar a arquitetura core. Ex: migrar DB para RDS.	Médio-Baixo	Ganhos rápidos sem refactoring completo
Repurchase (Drop & Shop)	Substituir por SaaS. Ex: CRM on-prem → Salesforce.	Médio	Software comercial com equivalente SaaS melhor
Refactor (Re-architect)	Redesenhar a aplicação para ser cloud-native (microserviços, serverless).	Alto	Necessidade de escala, agilidade, features cloud-native
Retire	Desligar aplicações que não são mais necessárias.	Baixo	Apps redundantes, sem uso, ou substituídos
Retain (Revisit)	Manter on-premises por enquanto. Migrar depois.	Nenhum	Compliance, dependências complexas, não pronto

Rehost

LIFT & SHIFT

Ferramentas:

- AWS Application Migration Service (MGN)
 - Replicação contínua de servidores
 - Cutover com mínimo downtime
 - Suporta Windows e Linux
- VM Import/Export
- CloudEndure Migration (predecessor do MGN)

Processo:

1. Instalar agente no servidor source
2. Replicação contínua para AWS
3. Test instances (validação)
4. Cutover (migração final)
5. Desligar source

Vantagens:

- Mais rápido de executar
- Menor risco (sem mudanças no app)
- Pode otimizar depois (replatform/refactor)

Replatform **LIFT & RESHAPE**

Exemplos comuns:

- MySQL on-prem → Amazon RDS MySQL
- Self-managed Redis → ElastiCache
- Apache on EC2 → Elastic Beanstalk
- Monolith on EC2 → ECS (containerizar)
- Cron jobs → EventBridge + Lambda

Benefícios sem refactoring completo:

- Managed services (menos operação)
- Auto-scaling, backups automáticos
- Multi-AZ (alta disponibilidade)
- Patching automático

Quando escolher:

- App funciona bem mas infra é pesada
- Quer benefícios de managed services
- Não tem tempo/budget para refactor completo

Refactor

RE-ARCHITECT

Padrões de modernização:

- Monolith → Microserviços
- Synchronous → Event-driven (SQS, SNS, EventBridge)
- Relational → Purpose-built databases
- Batch → Real-time streaming (Kinesis)
- On-prem storage → S3 data lake

Tecnologias cloud-native:

- Containers: ECS/EKS + Fargate
- Serverless: Lambda + API Gateway + DynamoDB
- Event-driven: EventBridge + Step Functions
- CI/CD: CodePipeline + CodeBuild + CodeDeploy

Quando escolher:

- Necessidade de escala massiva
- Time-to-market é prioridade
- Custos operacionais muito altos
- Features que só existem na cloud

Serviços de Migração

AWS Migration Hub **CENTRAL**

Painel centralizado para rastrear o progresso de migrações de múltiplas ferramentas.

Funcionalidades:

- Dashboard unificado de migração
- Rastreia progresso de múltiplas ferramentas
- Integra com: MGN, DMS, Application Discovery
- Strategy Recommendations: sugere estratégia (6 Rs)
- Refactor Spaces: gerencia refactoring incremental

Migration Hub Orchestrator:

- Workflows pré-definidos para migração
- Templates para SAP, SQL Server, etc.
- Automação de steps comuns

Application Discovery Service **DESCOBERTA**

Descobre servidores on-premises, dependências e padrões de uso para planejar migração.

Modos:

- Agentless (via VMware vCenter):
 - Informações básicas: CPU, memória, disco, rede
 - Sem instalar nada nos servidores
 - Bom para inventário inicial
- Agent-based:
 - Informações detalhadas + dependências
 - Processos rodando, conexões de rede
 - Mapa de dependências entre servidores
 - Necessário para planejamento detalhado

Integração:

- Migration Hub (visualização)
- Athena (queries nos dados coletados)
- QuickSight (dashboards de inventário)

AWS DMS DATABASE MIGRATION

Migra bancos de dados para AWS com mínimo downtime. Suporta migrações homogêneas e heterogêneas.

Tipos de migração:

- Homogênea: MySQL → RDS MySQL (mesmo engine)
- Heterogênea: Oracle → Aurora PostgreSQL (diferente)
 - Requer SCT (Schema Conversion Tool) primeiro

Modos:

- Full Load: copia todos os dados uma vez
- CDC (Change Data Capture): replicação contínua
- Full Load + CDC: copia tudo + mantém sincronizado

Características:

- Replication Instance: EC2 que executa a migração
- Source e Target endpoints
- Suporta: Oracle, SQL Server, MySQL, PostgreSQL, MongoDB, DynamoDB, S3, Redshift, Kinesis
- Multi-AZ para alta disponibilidade

AWS SCT (Schema Conversion Tool):

- Converte schema entre engines diferentes
- Identifica incompatibilidades
- Gera relatório de assessment
- Converte stored procedures, views, triggers

Transferência de Dados

AWS DataSync **TRANSFERÊNCIA**

Transferência de dados automatizada e acelerada entre on-premises e AWS (ou entre serviços AWS).

Características:

- Até 10x mais rápido que ferramentas open-source
- Agente on-premises (VM ou EC2)
- Criptografia em trânsito (TLS)
- Verificação de integridade automática
- Scheduling (transferências periódicas)
- Bandwidth throttling

Origens/Destinos:

- NFS, SMB, HDFS, self-managed object storage
- S3, EFS, FSx (Windows, Lustre, ONTAP, OpenZFS)
- Transferência entre serviços AWS (S3 → EFS)

Quando usar:

- Migração de dados para AWS
- Replicação periódica (backup)
- Transferência entre regiões/contas
- Mover dados entre storage services

vs Storage Gateway:

- DataSync: transferência/migração (mover dados)
- Storage Gateway: acesso híbrido contínuo (usar dados)

AWS Transfer Family **PROTOCOLOS**

Transferência de arquivos gerenciada via SFTP, FTPS, FTP e AS2 diretamente para S3 ou EFS.

Protocolos suportados:

- SFTP (SSH File Transfer Protocol)
- FTPS (FTP over SSL)
- FTP (File Transfer Protocol)
- AS2 (Applicability Statement 2)

Características:

- Endpoint público ou VPC
- Autenticação: service-managed, AD, custom (Lambda)
- Integra com S3 e EFS como backend
- IP estático (Elastic IP)
- Managed workflows (post-upload processing)

Quando usar:

- Parceiros/clientes enviam arquivos via SFTP
- Substituir servidor SFTP on-premises
- Compliance requer protocolos específicos
- B2B file exchange (AS2 para EDI)

AWS Snow Family OFFLINE

Dispositivos físicos para transferência massiva de dados quando a rede não é viável.

Dispositivos:

Snowcone	8 TB (HDD) ou 14 TB (SSD)
	Portátil, edge computing
	DataSync agent integrado
Snowball Edge	Storage Optimized: 80 TB
	Compute Optimized: 42 TB + GPU
	Edge computing (EC2, Lambda)
Snowmobile	100 PB (caminhão)
	Para migrações >10 PB
	Segurança física extrema

Regra de decisão:

- <10 TB ou boa conectividade → DataSync/Internet
- 10 TB - 10 PB → Snowball Edge
- >10 PB → Snowmobile
- Edge computing remoto → Snowcone/Snowball Edge

Processo:

1. Solicitar dispositivo no console
2. AWS envia o dispositivo
3. Copiar dados para o dispositivo
4. Enviar de volta para AWS
5. AWS carrega dados no S3

Direct Connect para Migração REDE DEDICADA

Quando usar DX para migração:

- Transferência contínua de grandes volumes
- Latência consistente necessária
- Migração de longa duração (meses)
- Workloads híbridos permanentes

Combinação com outros serviços:

- DX + DataSync: transferência acelerada via DX
- DX + DMS: migração de banco via conexão dedicada
- DX + Storage Gateway: acesso híbrido contínuo

Limitações para migração:

- Lead time: semanas a meses para provisionar
- Custo fixo mensal (porta + transferência)
- Não resolve migração inicial urgente
 - Use VPN como bridge temporária
 - Snow Family para bulk data inicial
 - DX para operação contínua depois

Modernização

Containerização

CONTAINERS

Estratégia de containerização:

1. Containerizar monolith (lift & shift para container)
2. Extrair serviços gradualmente
3. Migrar para orquestrador (ECS/EKS)
4. Adotar Fargate (serverless containers)

AWS App2Container:

- Ferramenta que containeriza apps Java/.NET
- Analisa dependências automaticamente
- Gera Dockerfile e task definitions
- Cria pipeline CI/CD

AWS Copilot:

- CLI para deploy de containers no ECS/Fargate
- Abstrai complexidade de ECS
- Environments, services, pipelines

Padrão Strangler Fig:

- Migrar funcionalidades incrementalmente
- Novo código em microserviços
- Roteamento gradual (API Gateway, ALB)
- Monolith diminui até ser desligado

Serverless MODERNIZAÇÃO

Padrão serverless completo:

- API: API Gateway + Lambda
- Data: DynamoDB ou Aurora Serverless
- Auth: Cognito
- Storage: S3
- Events: EventBridge + Step Functions
- Messaging: SQS + SNS

Benefícios:

- Zero gerenciamento de servidores
- Escala automática (0 → milhões)
- Paga apenas por uso
- Alta disponibilidade built-in

Quando modernizar para serverless:

- APIs com tráfego variável
- Event processing
- Scheduled tasks (cron)
- Data processing pipelines
- Microserviços com baixo tráfego

Decomposição de Microserviços **ARQUITETURA**

Padrões de decomposição:

- Por domínio de negócio (DDD - Bounded Contexts)
- Por subdomain (core, supporting, generic)
- Por verbo/ação (checkout, inventory, shipping)

Comunicação entre serviços:

- Síncrona: REST/gRPC (API Gateway, ALB)
- Assíncrona: eventos (SQS, SNS, EventBridge)
- Orquestração: Step Functions (workflow central)
- Coreografia: cada serviço reage a eventos

Padrões importantes:

- Saga Pattern: transações distribuídas
 - Orquestração (Step Functions)
 - Coreografia (eventos + compensação)
- CQRS: separar leitura de escrita
- Event Sourcing: estado como sequência de eventos
- Circuit Breaker: evitar cascading failures

Service Mesh (App Mesh):

- Comunicação service-to-service
- Observabilidade, retry, circuit breaker
- mTLS entre serviços

 **Para o exame:** "Migrar com mínimo downtime" → DMS com CDC + MGN. "Transferir petabytes" → Snow Family. "Migrar servidores" → Application Migration Service (MGN). "Descobrir dependências" → Application Discovery Service. "Modernizar monolith" → Strangler Fig + containers/serverless.

 SA Professional — Módulo 06

Palavras-Chave e Mapa de Serviços

Mapeamento rápido de palavras-chave para serviços AWS, árvores de decisão e comparações "quando usar X vs Y" para o exame.

Mapa de Palavras-Chave → Serviços

Quando ver estas palavras-chave no enunciado, pense imediatamente no serviço correspondente:

Compute

Palavra-chave no Enunciado	Serviço AWS
"Serverless", "sem gerenciar servidores", "event-driven"	Lambda
"Containers", "Docker", "microserviços"	ECS / EKS + Fargate
"Kubernetes", "portabilidade", "multi-cloud containers"	EKS
"Batch processing", "HPC", "jobs em lote"	AWS Batch
"Tolerante a interrupção", "90% desconto", "capacidade spare"	Spot Instances
"Workload estável", "compromisso 1-3 anos"	Reserved Instances / Savings Plans
"Licença por socket/core", "compliance de hardware"	Dedicated Hosts
"Deploy rápido de web apps", "sem gerenciar infra"	Elastic Beanstalk

Storage

Palavra-chave no Enunciado	Serviço AWS
"Object storage", "data lake", "backup", "ilimitado"	S3
"Arquivo", "compliance", "retenção longa", "vault"	S3 Glacier
"Shared file system", "NFS", "Linux", "containers"	EFS
"Windows file share", "SMB", "Active Directory"	FSx for Windows
"High performance", "HPC", "ML training", "Lustre"	FSx for Lustre
"Hybrid storage", "on-premises cache", "tape backup"	Storage Gateway
"Block storage", "EC2 volumes", "IOPS"	EBS
"Transferir dados", "migração de arquivos", "NFS/SMB → S3"	DataSync

Database

Palavra-chave no Enunciado	Serviço AWS
"Relacional", "MySQL/PostgreSQL compatível", "5x performance"	Aurora
"NoSQL", "key-value", "millisecond latency", "serverless DB"	DynamoDB
"Cache", "in-memory", "session store", "microsegundos"	ElastiCache (Redis)
"DynamoDB + microsegundos", "cache transparente"	DAX
"Graph", "relações", "social network", "fraud detection"	Neptune
"Time-series", "IoT metrics", "DevOps telemetry"	Timestream
"Ledger", "imutável", "auditável", "blockchain-like"	QLDB
"Data warehouse", "BI", "analytics pesado", "petabyte"	Redshift

Networking

Palavra-chave no Enunciado	Serviço AWS
"Conectar múltiplas VPCs", "hub central", "transitivo"	Transit Gateway
"Conexão dedicada", "baixa latência consistente", "1/10/100 Gbps"	Direct Connect
"Criptografado pela internet", "backup do DX", "rápido setup"	Site-to-Site VPN
"Acesso privado a serviços", "sem internet", "ENI na VPC"	PrivateLink / Interface Endpoint
"CDN", "cache na edge", "conteúdo estático/dinâmico"	CloudFront
"IPs estáticos globais", "TCP/UDP", "failover instantâneo"	Global Accelerator
"DNS", "routing policies", "health checks", "failover"	Route 53
"Rede global gerenciada", "política centralizada"	Cloud WAN

Security

Palavra-chave no Enunciado	Serviço AWS
"Multi-conta", "políticas de permissão máxima", "SCPs"	Organizations
"Landing zone", "guardrails", "account factory"	Control Tower
"SSO", "login único", "múltiplas contas"	IAM Identity Center
"Detecção de ameaças", "ML", "anomalias"	GuardDuty
"Compliance de configuração", "regras", "remediação"	AWS Config
"Auditoria de API", "quem fez o quê"	CloudTrail
"Visão centralizada de segurança", "findings agregados"	Security Hub
"Criptografia", "chaves gerenciadas", "rotation"	KMS
"WAF", "SQL injection", "XSS", "rate limiting"	AWS WAF
"DDoS", "proteção avançada", "SRT"	AWS Shield Advanced

Analytics & Integration

Palavra-chave no Enunciado	Serviço AWS
"Streaming", "real-time", "shards", "replay"	Kinesis Data Streams
"Entregar para S3/Redshift", "near real-time", "buffer"	Kinesis Data Firehose
"Query SQL em S3", "serverless analytics", "pay per scan"	Athena
"ETL", "data catalog", "crawlers", "schema discovery"	AWS Glue
"Fila de mensagens", "desacoplamento", "dead letter queue"	SQS
"Pub/sub", "fan-out", "múltiplos subscribers"	SNS
"Event bus", "SaaS events", "content filtering"	EventBridge
"Workflow", "state machine", "orquestração"	Step Functions

Migration

Palavra-chave no Enunciado	Serviço AWS
"Migrar servidores", "lift and shift", "replicação contínua"	Application Migration Service (MGN)
"Migrar banco de dados", "CDC", "replicação"	DMS
"Converter schema", "Oracle → PostgreSQL"	SCT
"Transferir petabytes", "offline", "dispositivo físico"	Snow Family
"Transferir arquivos", "NFS/SMB → S3/EFS"	DataSync
"SFTP gerenciado", "parceiros enviam arquivos"	Transfer Family
"Descobrir servidores", "dependências", "inventário"	Application Discovery Service

Árvores de Decisão

Escolha de Banco de Dados DECISÃO

Precisa de SQL/relacional?

- └─ Sim → Precisa de alta performance/escala?
 - └─ Sim → Aurora (MySQL/PostgreSQL)
 - └─ Não → RDS (MySQL, PostgreSQL, Oracle, SQL Server)
- └─ Não → Que tipo de dados?
 - └─ Key-value, alta escala → DynamoDB
 - └─ Grafos/relações → Neptune
 - └─ Time-series → Timestream
 - └─ Ledger/imutável → QLDB
 - └─ Wide-column (Cassandra) → Keyspaces
 - └─ Document (MongoDB) → DocumentDB
 - └─ Cache/session → ElastiCache Redis

Precisa de data warehouse?

- └─ Sim → Redshift (ou Athena para ad-hoc)

Escolha de Compute DECISÃO

Duração da execução?

- └─ <15 min, event-driven → Lambda
- └─ Longa duração, stateless → Containers (ECS/EKS + Fargate)
- └─ Longa duração, stateful → EC2

Precisa gerenciar infraestrutura?

- └─ Não → Fargate (containers) ou Lambda (functions)
- └─ Sim → EC2 ou ECS/EKS com EC2 launch type

Já usa Kubernetes?

- └─ Sim → EKS
- └─ Não → ECS (mais simples, AWS-native)

Batch processing?

- └─ Tolerante a interrupção → Spot Instances
- └─ Jobs em lote → AWS Batch
- └─ Big data → EMR

Escolha de Conectividade DECISÃO

Conectar on-premises → AWS?

└─ Precisa de baixa latência consistente?

| └─ Sim → Direct Connect

| └─ Não → Site-to-Site VPN

└─ Backup do Direct Connect?

| └─ VPN como failover

└─ Usuários remotos (laptops)?

└─ Client VPN

Conectar VPCs entre si?

└─ 2-3 VPCs, sem transitividade → VPC Peering

└─ 5+ VPCs ou transitividade → Transit Gateway

└─ Rede global complexa → Cloud WAN

Acessar serviços AWS sem internet?

└─ S3 ou DynamoDB → Gateway Endpoint (grátis)

└─ Outros serviços → Interface Endpoint (PrivateLink)

Performance global?

└─ Conteúdo web (cache) → CloudFront

└─ TCP/UDP, IPs estáticos → Global Accelerator

Escolha de Transferência de Dados

DECISÃO

Volume de dados?

- |— <10 TB → Internet/DataSync
- |— 10 TB - 10 PB → Snowball Edge
- |— >10 PB → Snowmobile

Transferência contínua ou única?

- |— Contínua → DataSync (scheduled) ou DX
- |— Única → Snow Family ou DataSync

Tipo de dado?

- |— Arquivos (NFS/SMB) → DataSync
- |— Banco de dados → DMS
- |— Parceiros enviam via SFTP → Transfer Family
- |— Streaming real-time → Kinesis

On-premises precisa acessar dados na AWS?

- |— Sim → Storage Gateway (cache local)

Quando Usar X vs Y

Transit Gateway vs VPC Peering NETWORKING

VPC Peering:

- ✓ 2-3 VPCs com comunicação direta
- ✓ Sem custo por hora (só data transfer)
- ✓ Menor latência (conexão direta)
- ✗ Não transitivo
- ✗ Não escala bem ($N \times (N-1) / 2$ conexões)

Transit Gateway:

- ✓ 5+ VPCs
- ✓ Conectividade transitiva
- ✓ Segmentação com route tables
- ✓ Integra com VPN e Direct Connect
- ✗ Custo por hora + data processing
- ✗ Latência ligeiramente maior (hop extra)

DynamoDB vs Aurora DATABASE

DynamoDB:

- ✓ Key-value / document access patterns
- ✓ Escala massiva (milhões de requests/s)
- ✓ Single-digit millisecond latency
- ✓ Serverless (zero admin)
- ✓ Global Tables (multi-region active-active)
- ✗ Sem joins, sem SQL complexo
- ✗ Queries limitadas (partition key + sort key)

Aurora:

- ✓ SQL complexo, joins, transações ACID
- ✓ Compatível MySQL/PostgreSQL
- ✓ Read replicas para offload
- ✓ Global Database (cross-region)
- ✓ Queries flexíveis (qualquer coluna)
- ✗ Escala vertical (instance types)
- ✗ Mais caro para alta throughput simples

SQS vs Kinesis

MESSAGING

SQS:

- ✓ Desacoplamento simples (producer/consumer)
- ✓ Mensagem deletada após processamento
- ✓ Escala automática ilimitada
- ✓ Dead Letter Queue para falhas
- ✓ Quando: task queue, buffering, 1 consumer group
- X Sem replay (mensagem some após consumo)
- X Sem ordering garantido (Standard)

Kinesis Data Streams:

- ✓ Múltiplos consumers leem mesmos dados
- ✓ Replay (re-processar dados passados)
- ✓ Ordering por shard (partition key)
- ✓ Retenção até 365 dias
- ✓ Quando: analytics real-time, múltiplos consumers
- X Gerenciar shards (capacity planning)
- X Mais complexo de operar

CloudFront vs Global Accelerator

PERFORMANCE

CloudFront:

- ✓ Cache de conteúdo (estático e dinâmico)
- ✓ HTTP/HTTPS
- ✓ Lambda@Edge / CloudFront Functions
- ✓ Quando: websites, APIs, streaming de vídeo
- ✗ Não suporta TCP/UDP genérico
- ✗ IPs mudam (não pode fazer whitelist)

Global Accelerator:

- ✓ TCP/UDP (qualquer protocolo)
- ✓ 2 IPs Anycast estáticos (whitelist)
- ✓ Failover instantâneo entre regiões
- ✓ Quando: gaming, IoT, VoIP, non-HTTP
- ✓ Health checks com failover automático
- ✗ Sem cache
- ✗ Mais caro que CloudFront para HTTP

EventBridge vs SNS EVENTS

SNS:

- ✓ Fan-out simples e rápido
- ✓ Alta throughput
- ✓ Mobile push notifications
- ✓ SMS, Email
- ✓ Quando: notificações, fan-out para SQS queues
- ✗ Filtering limitado (atributos simples)
- ✗ Sem archive/replay

EventBridge:

- ✓ Content-based filtering avançado (JSON path)
- ✓ Schema Registry (descoberta automática)
- ✓ Archive & Replay de eventos
- ✓ Integração com SaaS (Zendesk, Datadog)
- ✓ Quando: event-driven architecture complexa
- ✗ Menor throughput que SNS
- ✗ Mais caro por evento

DataSync vs Storage Gateway DATA

DataSync:

- ✓ MOVER dados (migração, sync periódico)
- ✓ Alta velocidade (10x ferramentas open-source)
- ✓ Verificação de integridade
- ✓ Scheduling
- ✓ Quando: migrar NFS/SMB para S3/EFS
- ✗ Não fornece acesso contínuo on-premises

Storage Gateway:

- ✓ ACessar dados na AWS de on-premises
- ✓ Cache local para baixa latência
- ✓ Protocolos nativos (NFS, SMB, iSCSI)
- ✓ Quando: acesso híbrido contínuo
- ✗ Não é ferramenta de migração
- ✗ Throughput limitado pelo cache local

 **Dica final:** No exame, identifique as palavras-chave no enunciado ANTES de ler as opções. Isso direciona seu raciocínio para o serviço correto e evita confusão com distratores. Pratique associar palavras-chave → serviço até virar automático.

 SA Professional — Módulo 07

Cenários Práticos

Cenários no estilo do exame cobrindo os 4 domínios, com análise de requisitos e justificativa detalhada da resposta correta.

Cenário 1 — Complexidade Organizacional

Situação **DOMÍNIO 1**

Uma empresa multinacional possui 200+ contas AWS em 5 regiões. A equipe de rede precisa conectar todas as VPCs entre si e com 3 data centers on-premises via Direct Connect. Atualmente usam VPC Peering, mas a complexidade de gerenciamento está insustentável. Precisam de segmentação entre ambientes (prod, dev, staging) e visibilidade centralizada do tráfego.

Requisitos identificados:

- Conectividade transitiva entre 200+ VPCs
- Integração com Direct Connect (3 data centers)
- Segmentação de tráfego por ambiente
- Visibilidade e monitoramento centralizado
- Escala multi-região

Opções:

A **INCORRETA**

Manter VPC Peering e adicionar mais conexões conforme necessário. Usar VPC Flow Logs para monitoramento.

✗ Por que está errada:

- VPC Peering não é transitivo (200 VPCs = ~20.000 conexões)
- Não escala para este cenário
- Sem segmentação nativa
- Gerenciamento impossível nesta escala

B INCORRETA

Usar Transit Gateway em cada região com peering inter-região. Criar route tables separadas para segmentação.

✗ Por que está errada:

- Transit Gateway resolve parcialmente
- Mas gerenciar TGW peering entre 5 regiões manualmente é complexo e não oferece política centralizada
- Falta visibilidade unificada
- Não é a solução mais operacionalmente eficiente

C ✓ CORRETA

Implementar AWS Cloud WAN com network policy centralizada. Definir segmentos (prod, dev, staging) na política. Integrar Direct Connect via Transit Gateway attachments.

✓ Por que está correta:

- Cloud WAN: rede global gerenciada como serviço
- Política centralizada define segmentação
- Integra automaticamente com DX via TGW
- Dashboard unificado de monitoramento
- Escala para centenas de VPCs em múltiplas regiões
- Menor esforço operacional para este cenário

D INCORRETA

Criar uma VPC compartilhada (shared VPC) usando AWS RAM e colocar todos os recursos nela.

✗ Por que está errada:

- Shared VPC não resolve conectividade entre VPCs existentes
- Não é viável migrar 200+ contas para shared subnets
- Não resolve conectividade com on-premises
- Viola princípio de isolamento entre contas

Cenário 2 — Design para Novas Soluções

Situação **DOMÍNIO 2**

Uma fintech precisa projetar um sistema de processamento de transações que recebe milhões de eventos por dia de múltiplas fontes (mobile, web, parceiros). Os dados precisam ser processados em tempo real para detecção de fraude, armazenados em um data lake para analytics, e disponibilizados em dashboards para a equipe de negócios. O sistema deve ter alta disponibilidade e custo otimizado.

Requisitos identificados:

- Ingestão de milhões de eventos/dia em tempo real
- Processamento em tempo real (detecção de fraude)
- Armazenamento em data lake para analytics
- Dashboards para equipe de negócios
- Alta disponibilidade e custo otimizado

Opções:

A **INCORRETA**

Usar SQS para ingestão, Lambda para processamento, RDS para armazenamento e QuickSight para dashboards.

✗ Por que está errada:

- SQS não permite múltiplos consumers lerem mesmos dados
- Mensagem é deletada após consumo (sem replay)
- RDS não é ideal para data lake (escala limitada)
- Não atende requisito de analytics em escala
- Funciona para desacoplamento, não para streaming

B ✓ **CORRETA**

Kinesis Data Streams para ingestão, Lambda/Kinesis Analytics para detecção de fraude em real-time, Kinesis Data Firehose para entregar dados no S3 (data lake), Glue para catalogação, Athena para queries ad-hoc, e QuickSight para dashboards.

✓ Por que está correta:

- Kinesis Streams: ingestão real-time, múltiplos consumers
- Lambda/Analytics: processamento em tempo real (fraude)
- Firehose → S3: entrega automática para data lake
- Glue Catalog + Athena: analytics serverless
- QuickSight: dashboards para negócios
- Tudo serverless = custo otimizado + HA built-in
- Arquitetura completa de streaming + data lake

C **INCORRETA**

Usar Apache Kafka em EC2 (MSK) para ingestão, EMR com Spark Streaming para processamento, HDFS para armazenamento e Kibana para dashboards.

✗ Por que está errada:

- MSK + EMR + EC2: alto esforço operacional
- HDFS não é ideal (S3 é melhor para data lake na AWS)
- Kibana não é serviço AWS nativo (OpenSearch Dashboards)
- Over-engineered para o cenário
- Custo mais alto (instâncias 24/7)
- Não segue princípio de "menor esforço operacional"

D **INCORRETA**

API Gateway + Lambda para ingestão, DynamoDB Streams para processamento, DynamoDB para armazenamento e QuickSight para dashboards.

✗ Por que está errada:

- API Gateway tem limite de 10.000 req/s (throttling)
- DynamoDB não é ideal para data lake analytics
- DynamoDB Streams tem retenção de apenas 24h
- Não escala para milhões de eventos/dia de forma custo-efetiva
- Analytics em DynamoDB é limitado e caro (scans)

Cenário 3 — Melhoria Contínua

Situação **DOMÍNIO 3**

Uma empresa de e-commerce tem uma aplicação rodando em EC2 com RDS MySQL. Durante a Black Friday, o banco de dados fica sobrecarregado com leituras (catálogo de produtos consultado milhões de vezes). A equipe quer melhorar a performance sem mudar a aplicação significativamente, mantendo custos controlados. O catálogo muda apenas a cada 5 minutos.

Requisitos identificados:

- Reduzir carga de leitura no RDS MySQL
- Dados mudam a cada 5 minutos (tolerância a stale data)
- Mínima mudança na aplicação
- Custo controlado
- Suportar picos de Black Friday

Opções:

A INCORRETA

Migrar para Aurora MySQL com 15 read replicas e usar o Reader Endpoint para distribuir leituras.

✗ Por que está errada:

- Resolve parcialmente (mais capacidade de leitura)
- Mas 15 replicas é caro (paga por instância)
- Ainda atinge o banco para cada request
- Não aproveita o fato de dados mudarem a cada 5 min
- Mudança significativa (migração de banco)

B ✓ CORRETA

Adicionar ElastiCache Redis na frente do RDS com TTL de 5 minutos. Implementar lazy loading: verificar cache primeiro, se miss, consultar RDS e popular cache.

✓ Por que está correta:

- Cache absorve 95%+ das leituras repetitivas
- TTL de 5 min alinhado com frequência de mudança
- Lazy loading: mudança mínima na aplicação
- Redis escala horizontalmente (cluster mode)
- Custo muito menor que 15 read replicas
- Latência de microsegundos vs milissegundos do RDS
- Padrão comprovado para catálogos de produtos

C INCORRETA

Escalar verticalmente o RDS para uma instância maior (db.r6g.16xlarge) e habilitar Performance Insights.

✗ Por que está errada:

- Scaling vertical tem limite (maior instância disponível)
- Custo muito alto para instância máxima
- Não resolve o problema fundamental (mesmas queries repetidas)
- Performance Insights é monitoramento, não solução
- Downtime durante resize (se não Multi-AZ)

D INCORRETA

Migrar o catálogo para DynamoDB com DAX para cache automático.

✗ Por que está errada:

- Migrar de MySQL para DynamoDB é mudança significativa
- Requer reescrever queries (SQL → NoSQL API)
- Catálogo pode ter relações complexas (categorias, variações)
- Viola requisito de "mínima mudança na aplicação"
- DAX é excelente, mas a migração é o problema

Cenário 4 — Migração e Modernização

Situação DOMÍNIO 4

Uma empresa precisa migrar um banco Oracle on-premises (5 TB) para AWS. O banco usa stored procedures complexas, triggers e packages específicos do Oracle. A empresa quer reduzir custos de licenciamento e adotar um banco open-source. O downtime máximo aceitável é de 2 horas durante a migração final.

Requisitos identificados:

- Migração heterogênea: Oracle → open-source (PostgreSQL)
- 5 TB de dados
- Stored procedures, triggers e packages Oracle-specific
- Downtime máximo: 2 horas
- Reduzir custos de licenciamento

Opções:

A **✗** INCORRETA

Usar apenas AWS DMS com full load para migrar os dados do Oracle para Aurora PostgreSQL em uma janela de manutenção.

✗ Por que está errada:

- Full load de 5 TB pode levar mais de 2 horas
- DMS sozinho NÃO converte stored procedures/triggers
- Schema incompatível (Oracle → PostgreSQL)
- Sem CDC, qualquer dado escrito durante migração é perdido
- Não aborda a conversão de código PL/SQL → PL/pgSQL

B **✓** CORRETA

1) Usar AWS SCT para converter schema, stored procedures e triggers de Oracle para PostgreSQL. 2) Resolver incompatibilidades manualmente identificadas pelo SCT. 3) Usar DMS com Full Load + CDC para replicar dados continuamente. 4) Cutover final (parar writes no Oracle, aguardar CDC sync, apontar app para Aurora PostgreSQL) dentro da janela de 2h.

✓ Por que está correta:

- SCT converte schema + identifica incompatibilidades
- Conversão manual apenas do que SCT não consegue
- DMS Full Load + CDC: replica dados continuamente
- Durante CDC, Oracle continua operando normalmente
- Cutover final é rápido (apenas parar writes + sync final)
- 2h é suficiente para cutover com CDC ativo
- Abordagem padrão AWS para migração heterogênea

C INCORRETA

Migrar para RDS Oracle primeiro (rehost), depois converter para Aurora PostgreSQL usando DMS.

✗ Por que está errada:

- Duas migrações = mais risco e mais tempo
- RDS Oracle ainda tem custo de licenciamento
- Não resolve o problema de conversão de stored procedures
- DMS entre RDS Oracle e Aurora ainda precisa de SCT
- Abordagem desnecessariamente complexa

D INCORRETA

Exportar o banco Oracle para dump files, transferir via Snowball Edge para S3, e importar no Aurora PostgreSQL.

✗ Por que está errada:

- Dump Oracle não é compatível com PostgreSQL diretamente
- Snowball Edge: lead time de dias (envio físico)
- Não resolve conversão de stored procedures
- Downtime seria muito maior que 2 horas
- Sem CDC = dados escritos durante transferência são perdidos
- Abordagem offline não atende requisito de downtime

Cenário 5 — Disaster Recovery (Domínio 2)

Situação **DOMÍNIO 2**

Um banco digital tem sua aplicação principal em us-east-1. Regulamentação exige que em caso de desastre regional, o sistema deve estar operacional em outra região em no máximo 5 minutos (RTO) com perda de dados máxima de 1 segundo (RPO). O banco processa transações financeiras 24/7 e não pode ter inconsistência de dados. O custo é secundário em relação à disponibilidade.

Requisitos identificados:

- RTO: 5 minutos

- RPO: 1 segundo
- Transações financeiras (consistência crítica)
- Operação 24/7
- Custo é secundário

Opções:

A **INCORRETA**

Warm Standby em us-west-2 com Aurora read replica cross-region e ASG com capacidade mínima. Route 53 failover com health checks.

✗ Por que está errada:

- Warm Standby pode atender RTO de 5 min
- MAS Aurora cross-region replica tem lag de ~1s (OK para RPO)
- Problema: scale up do ASG pode levar mais de 5 min
- Promoção de Aurora replica + DNS propagation
- Risco de não atender RTO consistentemente
- Não é a opção mais segura para requisitos tão rigorosos

B ✓ **CORRETA**

Multi-Site Active-Active em us-east-1 e us-west-2. Aurora Global Database com write forwarding. DynamoDB Global Tables para session/state. Route 53 latency-based routing com health checks. Infraestrutura completa em ambas as regiões.

✓ Por que está correta:

- Active-Active: ambas regiões processam tráfego
- RTO ~0: failover é apenas redirecionar tráfego (DNS)
- RPO <1s: Aurora Global Database replica em <1s
- DynamoDB Global Tables: multi-region active-active
- Route 53 health checks: failover automático
- Sem scale up necessário (infra já está rodando)
- Custo alto, mas requisito diz "custo é secundário"
- Atende todos os requisitos com margem de segurança

C INCORRETA

Pilot Light em us-west-2 com Aurora Global Database e AMIs pré-configuradas. Usar AWS Elastic Disaster Recovery para orquestrar failover.

✗ Por que está errada:

- Pilot Light: compute está DESLIGADO
- Iniciar instâncias + load balancer + warm up > 5 min
- Elastic DRS ajuda mas não garante RTO de 5 min
- Para banco digital 24/7, Pilot Light é arriscado
- RPO pode ser atendido (Aurora Global), mas RTO não

D INCORRETA

Backup & Restore com AWS Backup cross-region. RDS snapshots copiados a cada hora. CloudFormation para recriar infraestrutura na região DR.

✗ Por que está errada:

- RTO de horas (restaurar backups + provisionar infra)
- RPO de 1 hora (snapshots a cada hora)
- Completamente inadequado para os requisitos
- Backup & Restore é para workloads não-críticos
- Não atende nem RTO nem RPO exigidos

Cenário 6 — Governança Multi-Conta (Domínio 1)

Situação **DOMÍNIO 1**

Uma empresa está expandindo rapidamente e precisa criar novas contas AWS para cada equipe de produto. Requisitos: todas as contas devem ter CloudTrail habilitado, não podem criar recursos fora de us-east-1 e eu-west-1, Security Groups não podem ter ingress 0.0.0.0/0, e novas contas devem ser provisionadas com configuração padronizada automaticamente.

Requisitos identificados:

- Criação padronizada de contas (automática)
- CloudTrail obrigatório em todas as contas

- Restrição de regiões (us-east-1 e eu-west-1 apenas)
- Bloquear Security Groups abertos (0.0.0.0/0)
- Compliance contínuo e detectivo

Opções:

A **INCORRETA**

Usar AWS Organizations com SCPs para restringir regiões. Criar contas manualmente e configurar CloudTrail e Config em cada uma via CloudFormation StackSets.

✗ Por que está errada:

- Criação manual de contas não é padronizada
- StackSets resolve parcialmente, mas sem automação
- Falta Account Factory para padronização
- Falta guardrails detectivos automáticos
- Funciona, mas não é a solução mais completa/eficiente

B ✓ **CORRETA**

Implementar AWS Control Tower com: Account Factory para criação padronizada, Guardrails preventivos (SCPs) para restrição de regiões e bloqueio de desabilitar CloudTrail, Guardrails detectivos (Config Rules) para Security Groups abertos, e Organization Trail para CloudTrail centralizado.

✓ Por que está correta:

- Control Tower: solução completa para governança
- Account Factory: criação padronizada e automatizada
- Preventive Guardrails (SCPs):
 - Deny criar recursos fora de regiões permitidas
 - Deny desabilitar CloudTrail
- Detective Guardrails (Config Rules):
 - Detectar SGs com 0.0.0.0/0
 - Auto-remediation possível
- Organization Trail: CloudTrail em todas as contas
- Tudo integrado e gerenciado centralmente
- Novas contas herdam guardrails automaticamente

C INCORRETA

Criar um Lambda que monitorea CloudTrail events e reverte ações não-compliance. Usar EventBridge para trigger quando recursos são criados fora das regiões permitidas.

✗ Por que está errada:

- Abordagem reativa (permite e depois reverte)
- Janela de não-compliance entre criação e remediação
- Complexo de manter (Lambda custom code)
- Não padroniza criação de contas
- Não escala bem (cada regra = mais código)
- Reinventa o que Control Tower + SCPs já fazem

D INCORRETA

Usar IAM Policies em cada conta para restringir regiões e configurações. Criar um pipeline CI/CD que aplica as policies em novas contas.

✗ Por que está errada:

- IAM Policies podem ser modificadas por admins da conta
- SCPs são mais fortes (não podem ser overridden pela conta)
- Pipeline CI/CD é frágil (pode falhar, ser ignorado)
- Não há garantia de enforcement
- Falta visibilidade centralizada de compliance
- Não é a abordagem AWS recomendada

 Resumo dos padrões de resposta:

- Cenários de rede complexa → Cloud WAN ou Transit Gateway (depende da escala)
- Streaming + analytics → Kinesis + S3 data lake + Athena/QuickSight
- Performance de leitura → ElastiCache/DAX (cache é quase sempre a resposta)
- Migração heterogênea de banco → SCT + DMS com CDC
- DR com RTO ~0 → Multi-Site Active-Active
- Governança multi-conta → Control Tower + SCPs + Config Rules

